

Projeto de Banco de Dados

Kauan Henrique da Silva Rodrigues - 221017050

Lucas Vasconcelos Pirani Carneiro - 251031785

Yuri de Souza Neves - 211055960

Link do GitHub - https://github.com/BD-UNB/projeto_bd

1 Introdução

Durante os anos na UnB, observa-se dificuldade dos alunos para acessar informações sobre projetos de iniciação científica, estágio, monitoria, extensão e voluntariado, que ficam espalhadas em diversos canais. Isso dificulta a descoberta de vagas e o contato com professores ou coordenadores.

O projeto propõe um portal unificado que centralize anúncios, permita fórum e comentários e ofereça contato direto com os responsáveis, com o objetivo de facilitar o acesso às oportunidades acadêmicas e aumentar a visibilidade das vagas.

2 Modelo Entidade-Relacionamento

Para fazer a modelagem conceitual foi utilizada a ferramenta gratuita [brModelo](#). O diagrama completo do modelo entidade-relacionamento pode ser consultado no Apêndice A.

2.1 Entidades

As principais entidades identificadas no modelo são:

- **Usuários:** Entidade genérica que armazena informações de autenticação e identificação dos participantes do sistema.
- **Aluno:** Especialização de Usuários, contendo informações acadêmicas como matrícula, curso, nível e áreas de interesse.
- **Professor:** Especialização de Usuários, responsável pela criação e gerenciamento das oportunidades acadêmicas.
- **Vagas/Oportunidades:** Representa as oportunidades disponibilizadas no sistema, contendo informações como título, descrição, modalidade, carga horária e número de vagas.
- **Universidade, Departamento, Curso e Disciplina:** entidades responsáveis pela representação da estrutura acadêmica institucional.
- **Candidatura:** Registra as inscrições realizadas pelos alunos nas vagas disponíveis.
- **Conversa e Mensagem:** Permite a comunicação entre os usuários da plataforma.
- **Comentário_Vaga:** Possibilita comentários e discussões relacionados às oportunidades.
- **Tipo Vaga:** Classifica as oportunidades em diferentes categorias.
- **Campus:** Representa as unidades físicas vinculadas às universidades.

2.2 Relacionamentos

Os principais relacionamentos presentes no modelo são:

- Um **Professor** é responsável por uma ou mais **Vagas/Oportunidades**.
- Um **Aluno** pode realizar candidaturas para diferentes vagas por meio da entidade associativa **Candidatura**.
- As **Vagas/Oportunidades** pertencem a uma **Universidade** e são classificadas por um **Tipo de Vaga**.
- Um **Professor** pertence a um **Departamento**, que por sua vez pertence a uma **Universidade**.
- Os **Cursos** estão associados aos **Departamentos** e são compostos por diversas **Disciplinas**.
- As **Disciplinas** podem possuir pré-requisitos e professores responsáveis.
- Os usuários podem participar de **Conversas**, que contêm diversas **Mensagens**.
- As **Vagas/Oportunidades** podem receber comentários registrados na entidade **Comentário_Vaga**.

2.3 Cardinalidades

As cardinalidades definem a quantidade mínima e máxima de ocorrências permitidas entre entidades relacionadas. Alguns exemplos relevantes do modelo são:

- Uma **Universidade** possui vários **Departamentos**, enquanto cada departamento pertence a uma única universidade (1:N).
- Um **Departamento** pode possuir vários **Professores**, mas cada professor está vinculado a apenas um departamento (1:N).
- Um **Professor** pode ser responsável por várias **Vagas/Oportunidades**, enquanto cada vaga possui um único professor responsável (1:N).
- Um **Aluno** pode candidatar-se a várias vagas e uma vaga pode receber candidaturas de diversos alunos (N:M), sendo essa associação representada pela entidade **Candidatura**.
- Uma **Conversa** pode conter diversas **Mensagens**, enquanto cada mensagem pertence a uma única conversa (1:N).
- Um **Curso** é composto por várias **Disciplinas** e uma disciplina pode estar associada a diferentes cursos (N:M).
- Uma **Vaga/Oportunidade** pode possuir vários comentários, enquanto cada comentário está associado a uma única vaga (1:N).

3 Modelo Relacional

O modelo relacional também foi feito usando o [brModelo](#) visto que a própria ferramenta oferece uma opção de conversão do modelo conceitual para o modelo relacional. Durante o processo de conversão foi definido os domínios que os atributos iriam possuir como: INT, VARCHAR, ENUM, DATE, DATETIME, BOOLEAN e LONGBLOB para armazenar dados binários. Além disso, aqui também foi definido as restrições (*constraints*) que os atributos poderiam ter como: NOT NULL, UNIQUE, PRIMARY KEY e FOREIGN KEY, o próprio brModelo permite escolhermos manualmente essas opções.

O diagrama completo do modelo relacional pode ser consultado no Apêndice [B](#).

3.1 Principais Tabelas

As tabelas do banco de dados podem ser agrupadas conforme suas funcionalidades:

- **usuario**: Armazena os dados básicos de autenticação e identificação dos usuários do sistema.
- **aluno** e **professor**: Especializações da tabela usuário, contendo informações específicas de cada perfil.
- **universidade**, **departamento**, **curso**, **disciplina** e **campus**: Representam a estrutura acadêmica das instituições.
- **vagas_oportunidades**: Registra as oportunidades acadêmicas disponibilizadas na plataforma.
- **tipo_vaga**: Classifica as oportunidades em categorias específicas.
- **candidatura**: Registra as inscrições realizadas pelos alunos nas oportunidades.
- **comentario_vaga**: armazena comentários realizados pelos usuários sobre as oportunidades.
- **conversa** e **mensagem**: Implementam o sistema de comunicação entre usuários.
- **responsavel_vaga**: Associa professores responsáveis às oportunidades.
- **conversa_usuario**: Controla os participantes de cada conversa.
- **inscricao**: Representa a matrícula de alunos em cursos.
- **faz**: Relaciona alunos às disciplinas cursadas.
- **disc_curso**: Relaciona disciplinas aos cursos.
- **curso_departamento**: Relaciona cursos aos departamentos responsáveis.
- **pre_requisito**: Implementa o relacionamento de pré-requisitos entre disciplinas.

3.2 Chaves Primárias e Estrangeiras

As chaves primárias foram definidas para identificar unicamente cada registro das tabelas, enquanto as chaves estrangeiras garantem a integridade referencial entre os relacionamentos.

As principais chaves primárias são:

- `idUsuario` em `usuario`;
- `idAluno` em `aluno`;
- `idProfessor` em `professor`;
- `idUniversidade` em `universidade`;
- `idDepartamento` em `departamento`;
- `idCurso` em `curso`;
- `idDisciplina` em `disciplina`;
- `idCampus` em `campus`;
- `idVagas` em `vagas_oportunidades`;
- `idTipoVaga` em `tipo_vaga`;
- `idComentario` em `comentario_vaga`;
- `idConversa` em `conversa`;
- `idMensagem` em `mensagem`.

Além disso, diversas tabelas associativas utilizam chaves primárias compostas, abaixo segue alguns exemplos:

- `candidatura(idVagas, idAluno)`;
- `responsavel_vaga(idVagas, idProfessor)`;
- `conversa_usuario(idUsuario, idConversa)`;

- `inscricao(idAluno, idCurso);`
- `faz(idAluno, idDisciplina);`
- `disc_curso(idDisciplina, idCurso);`
- `curso_departamento(idCurso, idDepartamento);`
- `pre_requisito(idDisciplina, idPreRequisito).`

As principais chaves estrangeiras utilizadas no modelo são:

- `idUniversidade` em `departamento`, `curso` e `campus`;
- `idDepartamento` em `professor`, `disciplina` e `vagas_oportunidades`;
- `idTipoVaga` e `idCampus` em `vagas_oportunidades`;
- `idUsuario` em `mensagem` e `comentario_vaga`;
- `idConversa` em `mensagem`;
- `idVagas` em `comentario_vaga`, `candidatura`, `responsavel_vaga` e `conversa`;
- `idAluno` em `candidatura`, `inscricao` e `faz`;
- `idProfessor` em `responsavel_vaga`;

As tabelas associativas utilizam simultaneamente suas chaves compostas como chaves estrangeiras para as tabelas relacionadas.

Dessa forma, o modelo relacional preserva todas as restrições de integridade identificadas no modelo conceitual, garantindo consistência dos dados e suporte adequado às funcionalidades da plataforma.

4 Script SQL

Depois de realizar a modelagem relacional o próprio [brModelo](#) oferece um *script* SQL para a criação das tabelas do banco de dados, contudo o grupo achou esse *script* gerado de baixa qualidade e decidiu escrever por conta própria o código para criação das tabelas. O script SQL que gerou o banco de dados está disponível no [GitHub](#).

5 Camada de Persistência

A camada de persistência, também conhecida como camada de mapeamento ou de acesso a dados, é um componente fundamental na arquitetura do sistema. Seu principal objetivo é isolar a lógica de negócio da forma como os dados são armazenados e recuperados do banco de dados. Essa separação garante que alterações na interface gráfica ou na lógica de negócio não impactem diretamente a forma de interação com o SGBD, e vice-versa. Caso o SGBD seja alterado, por exemplo, apenas a camada de persistência precisaria ser adaptada, mantendo as demais camadas intactas.

5.1 Linguagem e Framework Utilizados

Para a implementação da camada de persistência, foi utilizada a linguagem de programação **Python**. A comunicação com o banco de dados MySQL é realizada através do driver `mysql.connector`, que atua como um framework de conexão, fornecendo as funcionalidades necessárias para estabelecer e gerenciar a comunicação com o SGBD.

5.2 Comunicação com o Banco de Dados

A comunicação com o banco de dados é encapsulada dentro da camada de persistência, onde as consultas SQL são construídas e executadas. Essa abordagem centraliza o código SQL, facilitando a manutenção e a reutilização. A arquitetura do sistema adota o padrão de *repositories*, onde cada entidade ou conjunto de funcionalidades relacionadas possui um repositório dedicado. Esses repositórios contêm métodos que abstraem as operações CRUD e outras consultas específicas, interagindo diretamente com o banco de dados.

A conexão com o MySQL é estabelecida utilizando as credenciais de acesso (host, porta, nome do banco, usuário e senha), que são configuradas de forma segura através de variáveis de ambiente. Um exemplo de como a conexão é obtida e utilizada para inicializar o banco de dados pode ser observado no trecho de código responsável pela execução do script de criação das tabelas.

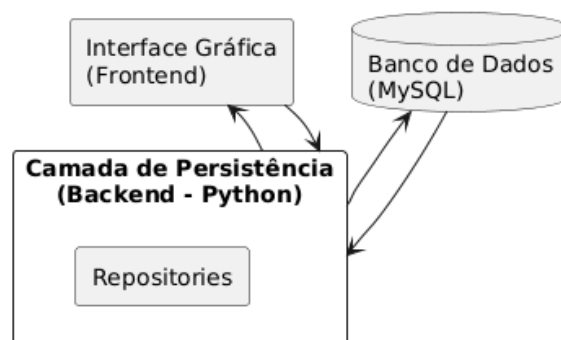


Figura 1: Diagrama da camada de persistência

5.3 Diagrama da Camada de Persistência

O diagrama da camada de persistência (Figura 1) ilustra como a interface gráfica do programa interage com a camada de persistência. A interface não acessa diretamente o banco de dados; em vez disso, ela envia requisições à camada de persistência, que é responsável por traduzir essas requisições em operações SQL e interagir com o SGBD. Essa arquitetura em camadas promove a modularidade e a manutenibilidade do sistema.

O código-fonte completo da camada de persistência, incluindo a configuração de conexão e os repositórios, está disponível no repositório do projeto no GitHub: [Link para o repositório GitHub](#).

6 Funcionalidades CRUD

As funcionalidades de Create, Read, Update e Delete (CRUD) são a base para a interação com os dados do sistema, permitindo a manipulação completa das informações armazenadas no banco de dados. Para demonstrar a implementação dessas operações, foram desenvolvidos exemplos em SQL que abrangem todas as tabelas do projeto.

6.1 Create (C)

A operação de criação de registros é realizada através do comando `INSERT INTO`. Este comando permite adicionar novas tuplas a uma tabela específica, preenchendo os valores

para suas colunas. Por exemplo, para adicionar uma nova universidade ou um novo aluno, o sistema executa um `INSERT` com os dados fornecidos. A ordem de inserção é crucial para tabelas com chaves estrangeiras, garantindo que os registros referenciados existam previamente.

6.2 Read (R)

A leitura de dados é implementada utilizando o comando `SELECT`. Esta operação permite recuperar informações de uma ou mais tabelas, com a possibilidade de aplicar filtros e ordenações. Para a demonstração, foram utilizados `SELECTs` simples para exibir todos os registros de uma tabela, bem como `SELECTs` com cláusulas `WHERE` para buscar registros específicos com base em critérios como `id` ou outros atributos.

6.3 Update (U)

A atualização de registros existentes é realizada por meio do comando `UPDATE`. Este comando permite modificar os valores de uma ou mais colunas em tuplas que satisfaçam uma condição específica definida pela cláusula `WHERE`. É fundamental utilizar a cláusula `WHERE` para evitar a atualização de todos os registros da tabela.

6.4 Delete (D)

A remoção de registros é efetuada com o comando `DELETE FROM`. Similar ao `UPDATE`, a cláusula `WHERE` é essencial para especificar quais registros devem ser removidos, prevenindo a exclusão acidental de todos os dados da tabela. A operação de `DELETE` deve ser executada com cautela, especialmente em tabelas que possuem relacionamentos de chave estrangeira, para evitar violações de integridade referencial.

6.5 Operações Envolvendo Múltiplas Tabelas

O sistema frequentemente realiza operações que envolvem a consulta e manipulação de dados em múltiplas tabelas simultaneamente. Isso é alcançado principalmente através de cláusulas `JOIN` em operações `SELECT`, permitindo combinar informações de tabelas relacionadas. Por exemplo, para listar vagas com detalhes do tipo de vaga e do departamento responsável, são utilizados `JOINS` entre `vagas_oportunidades`, `tipo_vaga` e `departamento`. Além disso, operações de `INSERT`, `UPDATE` e `DELETE` em tabelas associativas (como `candidatura`, `responsavel_vaga`, `inscricao`, etc.) naturalmente envolvem múltiplas tabelas, pois manipulam chaves estrangeiras que referenciam registros em outras entidades.

6.6 Garantia de Integridade Referencial

A integridade referencial é um pilar fundamental do projeto, assegurada pela definição de chaves estrangeiras (`FOREIGN KEY`) no modelo relacional. Conforme detalhado na Seção 3.2, cada relacionamento entre tabelas é protegido por uma chave estrangeira que garante que um valor em uma coluna de uma tabela corresponda a um valor em uma coluna de outra tabela.

Essa abordagem previne a inconsistência dos dados. Por exemplo, não é possível inserir uma candidatura para uma vaga que não existe, nem associar um professor a um

departamento inexistente. Ao tentar realizar operações que violam essas restrições, por exemplo deletar uma universidade que ainda possui departamentos associados sem uma política de exclusão em cascata, o SGBD (MySQL 8.0) impede a ação, retornando um erro de violação de chave estrangeira. Isso garante que o banco de dados mantenha um estado consistente e válido em todas as operações CRUD.

Exemplos detalhados de todas as operações CRUD para as tabelas do projeto podem ser encontrados no arquivo [crud.sql](#) no repositório do GitHub.

7 Recursos Avançados

7.1 Views

Foi criada a view `vw_vagas_com_contagem_candidatos` com o objetivo de consolidar informações relevantes sobre as oportunidades cadastradas na plataforma. A view reúne dados das tabelas de vagas, tipos de vaga, departamentos e candidaturas, permitindo visualizar em uma única consulta informações como título da vaga, modalidade, nível, departamento responsável e quantidade de candidatos inscritos.

A principal vantagem dessa abordagem é simplificar consultas frequentemente utilizadas pela aplicação e por relatórios administrativos, reduzindo a necessidade de múltiplos *joins* e operações de agregação. Além disso, a view pode servir como base para dashboards e funcionalidades de acompanhamento das oportunidades disponibilizadas no sistema.

O código completo da view encontra-se disponível no repositório do projeto no GitHub: [GitHub](#).

7.2 Triggers

Foi implementada a trigger `trg_atualiza_status_vaga_apos_candidatura`, responsável por automatizar a atualização do estado das oportunidades cadastradas na plataforma. Sempre que uma nova candidatura é registrada, a trigger verifica o status atual da vaga correspondente e, caso ela esteja marcada como **publicada**, seu status é automaticamente alterado para **em_analise**.

Essa automatização reduz intervenções manuais por parte dos responsáveis pelas vagas, mantém a consistência das informações armazenadas e permite que o sistema reflita de forma imediata o andamento do processo seletivo.

O código completo da trigger encontra-se disponível no repositório do projeto no GitHub: [GitHub](#).

7.3 Procedures

Foi implementada a procedure `ListarVagasPublicadas` com o objetivo de fornecer uma rotina parametrizável para recuperar as oportunidades com status **publicada**. A procedure retorna informações completas das vagas presentes na tabela `vagas_oportunidades` e inclui dados relacionados de tipo de vaga e departamento, permitindo à aplicação e aos relatórios obterem uma visão consistente e direta das oportunidades publicadas.

A procedure recebe dois parâmetros opcionais: `p_idTipoVaga` e `p_idDepartamento`. Quando um parâmetro é informado a consulta filtra as vagas pelo respectivo valor; quando o parâmetro é `NULL` o filtro é omitido, permitindo tanto consultas gerais quanto buscas

específicas sem necessidade de múltiplas queries. A rotina realiza JOIN com as tabelas `tipo_vaga` e `departamento` para incluir nomes descritivos, aplica a condição fixa `vo.status = 'publicada'` para garantir que apenas vagas publicadas sejam retornadas e devolve os campos principais da vaga junto com os nomes de tipo e departamento.

O código completo da procedure encontra-se disponível no repositório do projeto no GitHub: [GitHub](#).

7.4 Dados Binários

Utilizamos o tipo BLOB para armazenar os arquivos PDF referentes ao atributo "currículo" dos alunos, garantindo que os documentos binários fiquem salvos diretamente no banco de dados. O uso de BLOB facilita controle transacional e backup centralizado, mas pode aumentar o tamanho do banco e impactar desempenho; por isso, avalia-se esse trade-off conforme requisitos de escalabilidade e recuperação.

8 Uso de Inteligência Artificial

Durante a elaboração deste relatório utilizamos ferramentas de Inteligência Artificial (IA) para apoiar tanto a estruturação do documento quanto a modelagem do banco de dados e a revisão dos artefatos produzidos.

Inicialmente, o ChatGPT foi usado para definir o formato e a estrutura do relatório. Em seguida, após a criação da primeira versão da modelagem conceitual, enviamos a imagem (Figura 2) ao ChatGPT e solicitamos recomendações de melhoria.

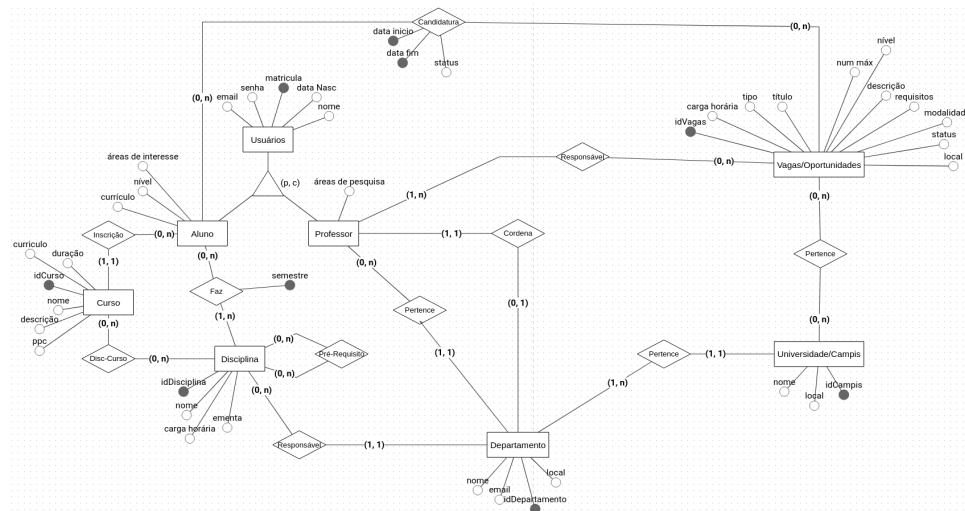


Figura 2: 1ª Versão do Modelo Conceitual

Dentre as várias sugestões recebidas, adotamos duas mudanças principais: separar a entidade **Universidade** da entidade **Campus**, e introduzir as entidades **Conversa** e **Mensagem** para armazenar o histórico de trocas entre usuários do sistema. Essas alterações foram incorporadas e podem ser vistas na versão atual do modelo conceitual apresentada anteriormente.

Ademais, um membro da equipe propôs uma adaptação na modelagem relacional para refletir melhor a estrutura acadêmica. Essa modificação também foi revisada com o auxílio

do ChatGPT, que vinculou a tabela `Curso` diretamente à `Universidade` e introduziu uma tabela associativa `curso_departamento`. A mudança garante que cada curso esteja corretamente associado tanto à sua universidade quanto aos departamentos responsáveis, representando de forma mais fiel a hierarquia institucional (`universidade` → `departamento` → `curso` → `disciplina`) e facilitando consultas e a organização dos dados.

Conforme descrito na Seção 4, optamos por escrever manualmente o código SQL de criação das tabelas. Com o script pronto, pedimos ao ChatGPT que analisasse o código junto com a imagem da modelagem relacional para identificar e corrigir possíveis erros de codificação e inconsistências entre o modelo e o script. O resultado dessa revisão foi consolidado no arquivo `script.sql`, disponível no repositório do projeto no GitHub: https://github.com/BD-UNB/projeto_bd/blob/main/database/script.sql.

Além das mudanças de modelagem e do ajuste do script SQL, a IA contribuiu com outras sugestões e protótipos avaliados pela equipe:

- Associação entre `Conversa` e `Vagas/Oportunidades`, permitindo registrar diálogos vinculados a processos seletivos;
- Implementação de comentários em páginas de vagas, para interação entre candidatos e recrutadores;
- Proposta de arquitetura para o chat entre alunos e professores (exemplo técnico em Python para backend e troca de mensagens);
- Recomendações para a página de administração, incluindo alterações no modelo de dados e orientações de cautela no backend quanto a permissões e validação de entrada.

Para o banco de dados foi utilizada a IA para popular o banco de dados com pelo menos cinco registros em cada tabela e também para gerar a trigger e a procedure com base no exemplo apresentado pela professora.

Para o frontend a IA foi utilizada para dicas de como criar as páginas de forma bem estruturada para não ficar muito poluída e nem faltar informações, além de consultas de como realizar chamadas de funções ou expressões que renderizam uma lista, como é possível ver a lista de vagas adicionadas em `home_admin` → gerenciar vagas.

Para o backend, houve a utilização de ferramenta de IA para estabelecer a melhor prática para lidar com sessões, em que foi sugerida a biblioteca `PyJWT`. Além disso, pediu-se a melhor prática para lidar com senhas e segurança, em que foi sugerida a biblioteca `bcrypt` para lidar com o login.

Por fim, salientamos que a IA também foi empregada na revisão linguística e correção ortográfica do texto deste relatório. Todas as sugestões geradas por ferramentas de IA foram avaliadas criticamente pela equipe; as alterações que impactam o comportamento do sistema ou a integridade do modelo de dados foram discutidas e aprovadas pelos responsáveis antes da implementação.

A Modelo Conceitual Completo

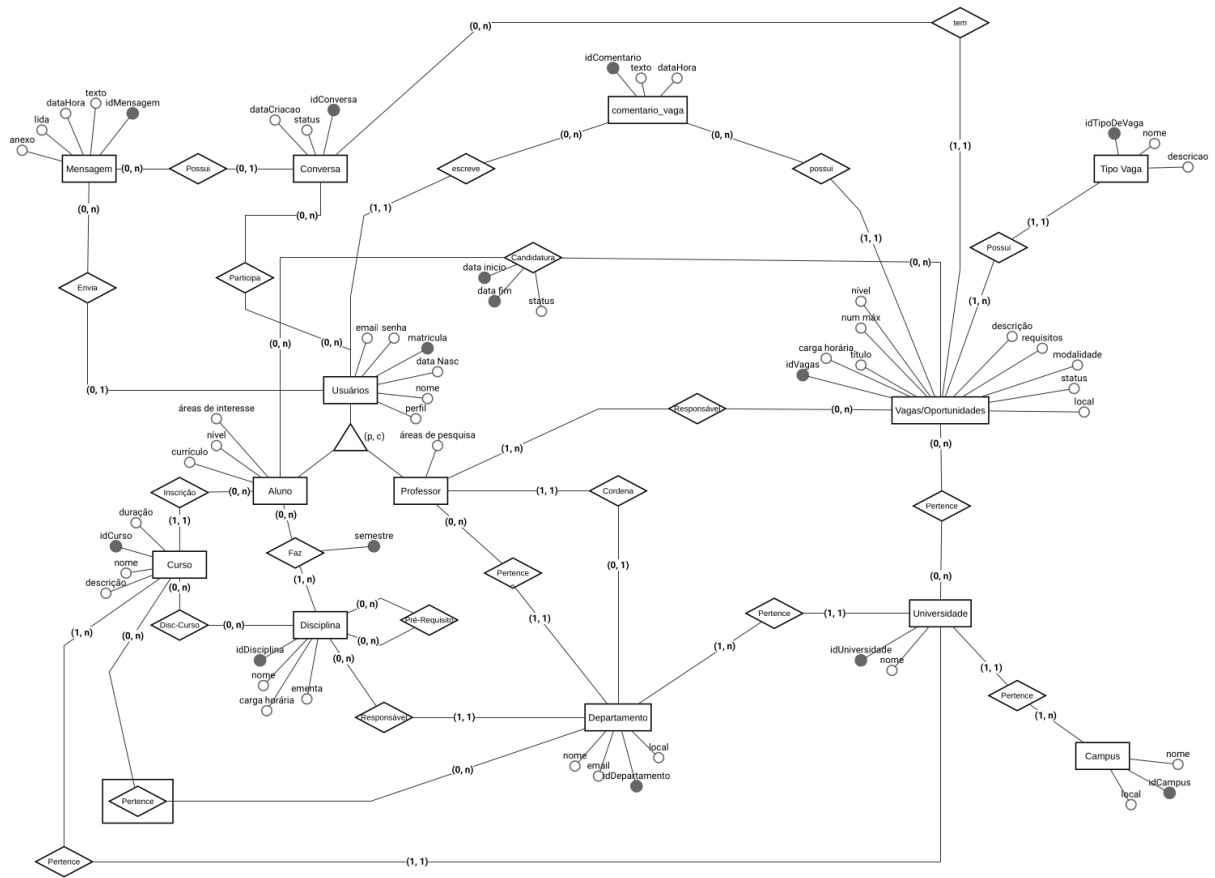


Figura 3: Modelo Conceitual Completo

B Modelo Relacional Completo

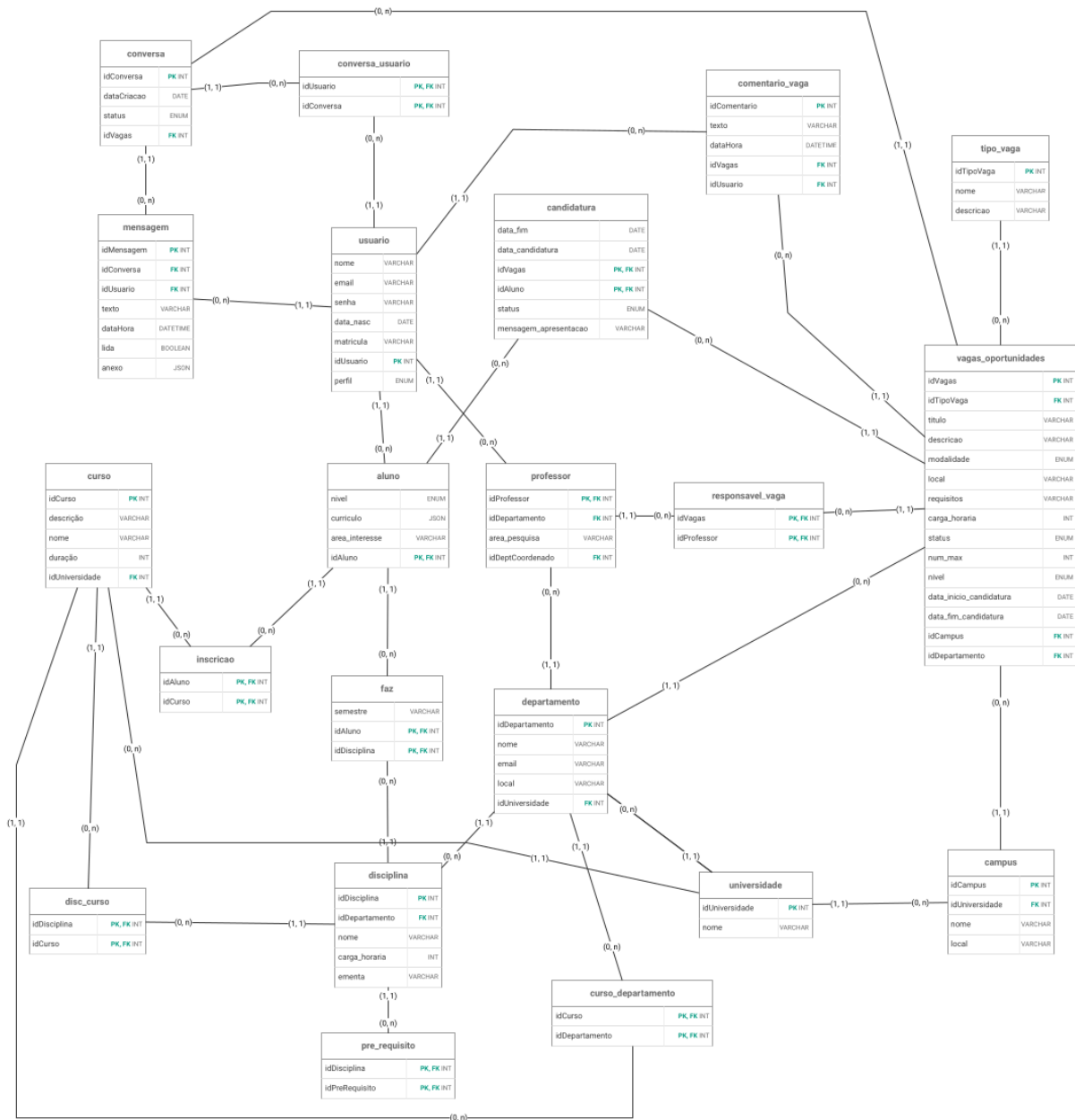


Figura 4: Modelo Relacional Completo